

Dossier Personnel

BTS CIEL SESSION 2025 - IR

E6 – PROJET TECHNIQUE

Pilotage d'éclairage scénique multi-univers Art-Net/DMX



Lycée : Touchard-Washington		Ville : LE MANS	
Nom du projet :	Pilotage d'éclairage scénique multi-univers Art-Net/DMX		
N° du projet :	TW3		
Projet nouveau	Oui ☒ Non ☐	Projet interne	Oui ☒ Non ☐
Délai de réalisation	150 heures	Statut des étudiants	Formation initiale ☒ Apprentissage ☐
Spécialité des étudiants	ER ☐ IR ☒ Mixte ☐	Nombre d'étudiants :	4 étudiants
Professeurs responsables	François MARTIN, Philippe CRUCHET, Didier BERNARD, Anthony LE CREN, Jilali KHAMLACH, François QUEREC		

HENRIETTE Naël (Étudiant 4)

Sommaire

Table des matières

1. Cas d'utilisation.....	3
1.1. Convertir Art-Net en DMX512.....	3
1.2. Piloter le ruban LED à partir du protocole DMX512.....	4
2. Diagrammes de séquences.....	5
2.1. Convertir Art-Net en DMX512.....	5
2.2. Piloter le ruban LED à partir du protocole DMX512.....	6
3. Protocoles.....	7
3.1. Protocole Art-Net (UDP).....	7
3.2. Protocole DMX512.....	8
3.3. Bus UART & RS-485 (Interface DMX).....	9
3.4. Protocole Wi-Fi (Norme IEEE 802.11).....	10
4. Outils.....	11
4.1. Visual Studio Code (VS Code).....	11
4.2. PlatformIO.....	12
4.3. Bibliothèques utilisées.....	13
4.3.1. FastLED (Gestion du ruban lumineux).....	13
4.3.2. Bibliothèque esp_dmx (Contrôleur RS-485).....	13
4.3.3. Bibliothèque SSD1306 (Interface Homme-Machine).....	14
4.3.4. WiFi et WiFiUDP (Connectivité réseau et Art-Net).....	14
4.3. Github.....	15
5. Diagrammes de classe.....	16
6. Fiches de test.....	18
7. Diagramme de Gantt.....	21
8. Compétences acquises.....	22
9. Perspectives d'améliorations.....	23

1.Cas d'utilisation

1.1.Convertir Art-Net en DMX512

Nom du cas 1 d'utilisation	Convertir Art-Net en DMX512
Pré-conditions :	- La passerelle (LaCarte) est connectée au réseau local et à l'écoute des paquets UDP . - Le module est configuré sur le bon Univers (ex: Univers 1, 2 ou 3) . - La liaison série (UART) vers l'adaptateur RS-485 est initialisée. - Les projecteurs DMX physiques sont raccordés au bus série RS485.
Post-conditions :	- Les projecteurs DMX connectés reçoivent un signal DMX512 valide et mettent à jour leur état lumineux.
Scénario principal :	1. Le module ESP32 écoute sur le port UDP standard Art-Net (6454). 2. Le Serveur Central (Raspberry Pi) envoie une trame Art-Net contenant les données d'éclairage. 3. L'ESP32 reçoit la trame et vérifie l'entête (Header "Art-Net"). 4. L'ESP32 extrait les 512 canaux de données (DMX data). 5. L'ESP32 convertit ces données en signal série et les transmet via l'adaptateur RS-485 aux projecteurs.
Alternatives :	- Si un délai critique de 3 secondes s'écoule sans qu'aucune trame de données ne soit correctement transmise ou acquittée 1. Le système enclenche un mécanisme de sécurité 2. Une fois le temps écoulé (3s écoulée), la passerelle signale la coupure de communication

1.2.Piloter le ruban LED à partir du protocole DMX512

Nom du cas d'utilisation	Piloter le ruban LED à partir du protocole DMX512
Pré-conditions :	- Le module ESP32 dédié au ruban LED est alimenté. - Le ruban LED est physiquement connecté aux broches PWM de l'ESP32. - Le module reçoit des instructions DMX (soit via le réseau Art-Net converti en interne, soit via un signal DMX filaire selon l'architecture choisie).
Post-conditions :	- Le ruban LED s'allume avec la couleur et l'intensité demandées.
Scénario principal :	1. Le module reçoit les valeurs DMX pour les 6 canaux (Rouge, Vert, Bleu, Zones, Intensité et Effets). 2. Le système associe les canaux DMX reçus aux broches de sortie correspondantes. 3. Le microcontrôleur génère des signaux PWM (Pulse Width Modulation) proportionnels aux valeurs reçues (0-255). 4. Le ruban LED change de couleur ou d'intensité instantanément.
Alternatives :	- Si un délai critique de 3 secondes s'écoule sans qu'aucune trame de données ne soit correctement transmise ou acquittée 1. Le système enclenche un mécanisme de sécurité 2. Une fois le temps écoulé (3s écoulée), la passerelle signale la coupure de communication

2. Diagrammes de séquences

2.1. Convertir Art-Net en DMX512

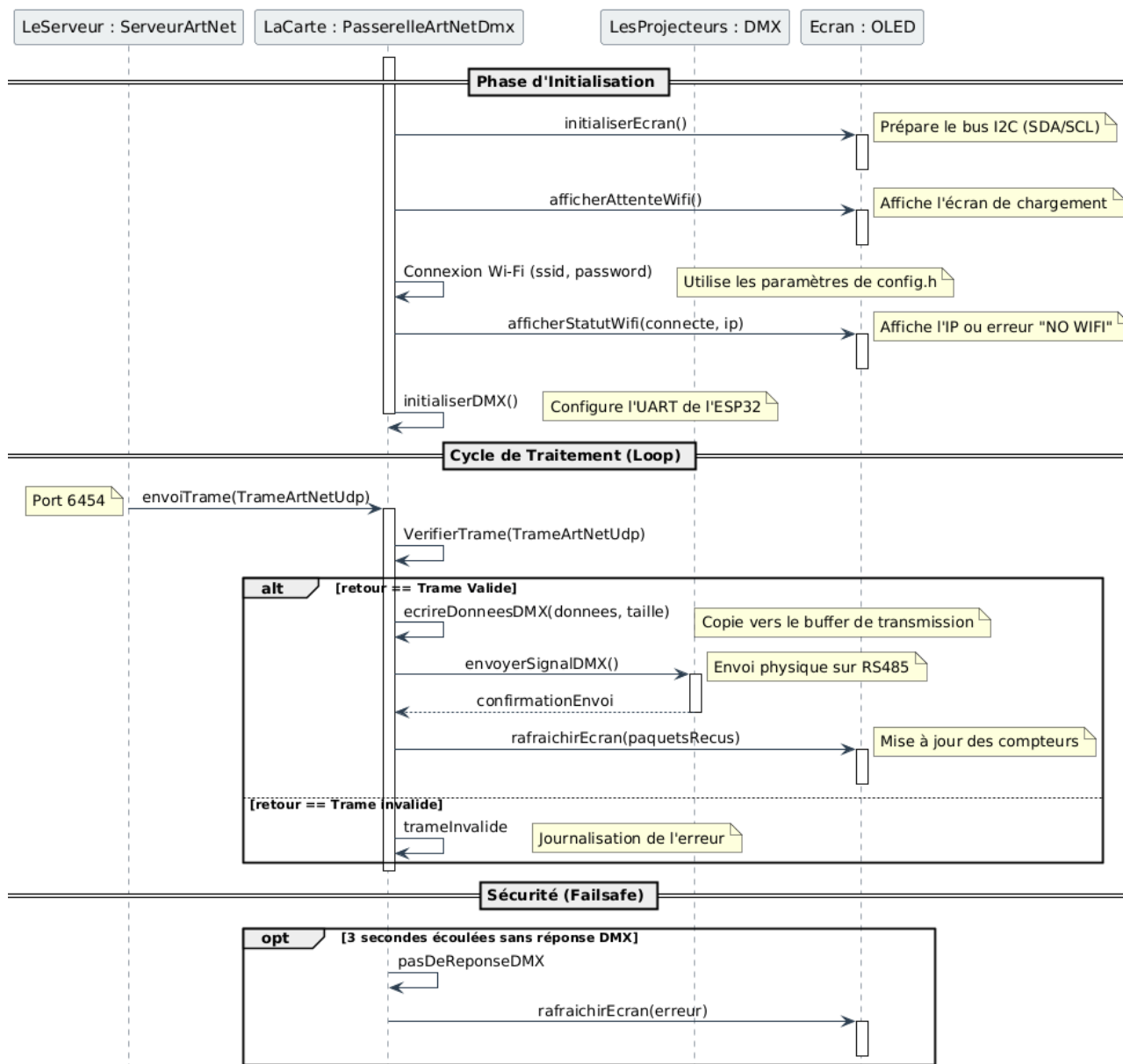


Image 1 : Diagramme de séquence cas d'utilisation 1

La passerelle reçoit et vérifie les trames UDP Art-Net. Si la trame est valide, elle l'envoie aux projecteurs DMX et met à jour l'écran OLED ; sinon, elle la rejette. En parallèle, une sécurité (Failsafe) surveille la liaison et affiche une alerte sur l'OLED après 3 secondes de perte de signal.

2.2. Piloter le ruban LED à partir du protocole DMX512

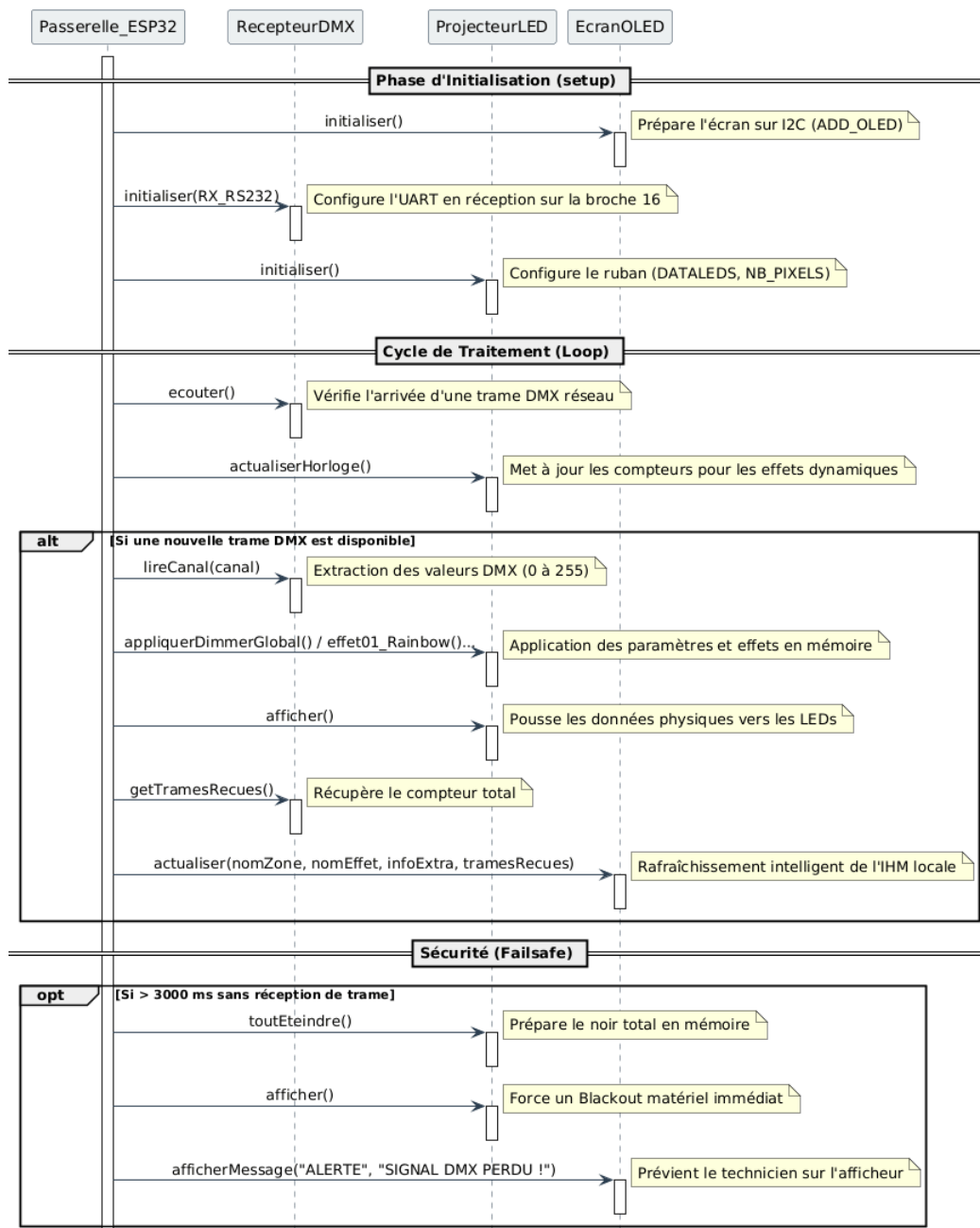


Image 2 : Diagramme de séquence cas d'utilisation 2

La passerelle écoute et décode les trames DMX entrantes. Si une trame est disponible, elle applique les effets lumineux sur le ruban LED et met à jour les statistiques sur l'écran OLED. En parallèle, une sécurité (Failsafe) surveille la liaison et force un blackout matériel couplé à une alerte sur l'OLED après 3 secondes de perte de signal.

3. Protocoles

3.1. Protocole Art-Net (UDP)

Art-Net est un protocole réseau développé par Artistic Licence en 1998. Il permet de transporter le signal DMX 512 sur un réseau Ethernet standard via UDP (User Datagram Protocol). Contrairement à TCP, UDP n'établit pas de connexion et ne garantit pas la livraison — ce qui est acceptable en éclairage scénique

Avantage principal : Art-Net permet de transmettre jusqu'à 32 768 univers DMX sur un seul réseau IP, là où le DMX classique se limite à 512 canaux par câble physique.

Un paquet Art-Net (OpDmx) se compose d'un en-tête fixe suivi des 512 valeurs de canaux DMX. Voici sa structure détaillée :

Octet(s)	Champ	Valeur	Description
0 — 7	ID	41 72 74 2D 4E 65 74 00	Signature ASCII "Art-Net" + '0'
8 — 9	OpCode	00 50	OpDmx : commande d'envoi DMX (little-endian)
10 — 11	ProtVer	00 0E	Version du protocole Art-Net (14)
12	Sequence	00	Séquence (0 = désactivée)
13	Physical	00	Port physique source
14 — 15	Universe	00 00	Numéro d'univers DMX (0 à 32767)
16 — 17	Length	02 00	Nombre de canaux DMX (512 = 0x0200)
18 — 529	Data	00 FF 00 80 ...	Valeurs des 512 canaux DMX (0 à 255)

Utilisation dans le projet

Dans le DMX Equipment Manager, chaque univers DMX est associé à une adresse IP correspondant au nœud Art-Net (boîtier convertisseur Ethernet/DMX ou Raspberry Pi). L'adresse IP et le numéro d'univers sont configurés par l'utilisateur et stockés en base de données.

Lorsqu'une scène est lancée, le Raspberry Pi récupère les valeurs des canaux et les envoie au nœud Art-Net correspondant sous forme de paquets UDP sur le port 6454 (port standard Art-Net).

3.2. Protocole DMX512

DMX 512 (Digital Multiplex with 512 pieces of information) est un protocole de communication numérique standardisé (ANSI E1.11) utilisé pour le contrôle des équipements d'éclairage scénique depuis 1986. Il permet de contrôler jusqu'à 512 canaux indépendants sur un seul câble, chaque canal pouvant prendre une valeur entre 0 et 255.

Chaque appareil DMX occupe un certain nombre de canaux consécutifs à partir de son adresse de départ. Par exemple, un projecteur à l'adresse 1 avec 4 canaux occupe les canaux 1, 2, 3 et 4.

Le signal DMX est transmis en série à 250 kbauds via RS-485. Une trame DMX complète se compose des éléments suivants :

Élément	Durée	Description
Break	$\geq 88 \mu\text{s}$	Signal bas indiquant le début d'une nouvelle trame
Mark After Break (MAB)	$\geq 8 \mu\text{s}$	Signal haut séparant le Break du premier octet
Start Code	1 octet (44 μs)	Code de démarrage, toujours 0x00 pour DMX standard
Slot 1 à 512	1 octet chacun	Valeur de chaque canal DMX (0 à 255)
Mark Time Between Frames	Variable	Temps entre deux trames (optionnel)

Voici une représentation simplifiée d'une trame DMX pour 4 canaux :

BREAK	→	_____
MAB	→	-----
Start Code	→	0x00 (DMX standard)
Canal 1	→	0xFF (255 – pleine intensité)
Canal 2	→	0x80 (128 – mi-intensité)
Canal 3	→	0x1A (26 – Rouge : 10%)
Canal 4	→	0x00 (0 – éteint)
...	→	0x00 (canaux 5 à 512 à 0)

Dans le DMX Equipment Manager, chaque équipement est enregistré avec son adresse DMX de départ et son nombre de canaux. L'application calcule automatiquement le canal absolu de chaque canal :

$\text{Canal absolu} = \text{adresseDepart} + \text{numeroCanal} - 1$

Exemple : Projecteur à l'adresse 5, canal 3
 → Canal absolu = 5 + 3 - 1 = 7

3.3. Bus UART & RS-485 (Interface DMX)

Lors de la communication avec les projecteurs de scène, notre système doit traduire les données logiques de l'ESP32 en signaux électriques robustes. Pour ce faire, nous utilisons le port série matériel asynchrone interne de la carte (UART) couplé à un convertisseur externe (MAX485). L'UART génère la trame de données, et le composant MAX485 la transforme en un signal électrique différentiel (norme RS-485). Cela garantit une immunité totale aux parasites électromagnétiques, même sur des câbles de plusieurs centaines de mètres.

Pour que ce signal soit reconnu comme du DMX512 valide par les projecteurs de l'industrie, la configuration temporelle du port série matériel est extrêmement stricte.

Format de configuration : {<Vitesse_Bauds>, <Bits_Données>, <Parité>, <Bits_Stop>, <Broche_TX>, <Broche_RX>}

Champs de configuration UART	Paramètre DMX	Valeur stricte
Vitesse de transmission	Baud Rate	250 000 bps (Cadencement standard)
Taille des données	Data Bits	8 bits (1 octet complet par canal)
Vérification d'erreur	Parity	Aucune (None)
Bits d'arrêt	Stop Bits	2 bits (Délimite la fin de l'octet)
Transmission (Vers MAX485)	TX_PIN	Broche 17 (Génération du signal)
Réception (Depuis MAX485)	RX_PIN	Broche 16 (Écoute du bus réseau)

Exemple : {"Baud_Rate":250000, "Data_Bits":8, "Parity":"None", "Stop_Bits":2, "TX_PIN":17, "RX_PIN":16}

(Cette configuration verrouille matériellement la communication série de l'ESP32. Elle garantit que les impulsions électriques envoyées à la puce MAX485 via la broche 17 respectent un rythme exact de 250 000 bits par seconde, avec un format de 8 bits de données encadrés par 2 bits de stop, répondant ainsi parfaitement aux exigences temporelles draconiennes du protocole DMX512).

3.4. Protocole Wi-Fi (Norme IEEE 802.11)

Pour permettre la réception en temps réel des trames Art-Net provenant du serveur (la régie lumière), notre passerelle ESP32 doit être intégrée au réseau local. Nous utilisons pour cela la norme de communication sans fil Wi-Fi en mode "Station" (STA). Dans cette configuration, le microcontrôleur agit comme un client standard qui s'authentifie auprès d'un routeur ou d'un point d'accès existant.

Une fois la connexion sécurisée établie, la carte se voit attribuer une adresse IPv4 locale. C'est cette adresse qui lui permet d'être identifiable sur le réseau et d'ouvrir son port d'écoute UDP.

La structure des informations requises pour l'initialisation de la couche réseau est la suivante.

Format de configuration : {<Nom_du_Réseau>, <Clé_de_Sécurité>, <Mode_Fonctionnement>, <Adresse_IP>}

Champs de configuration réseau	Nom (Code)	Valeur configurée ou attendue
Identifiant du réseau (SSID)	ssid	"Nom_Du_Reseau_Local"
Clé d'authentification (WPA2)	password	"*****"
Mode de connexion	WIFI_MODE	WIFI_STA (Mode Station)
Adresse réseau attribuée	localIP()	ex: 192.168.1.50 (Assignation DHCP ou Fixe)

Exemple de validation réseau : {"ssid":"Regie_Lumiere", "password":"****", "mode":"WIFI_STA", "IP":"192.168.1.50"}

(Cette configuration permet à l'ESP32 de s'authentifier de manière sécurisée sur l'infrastructure locale. L'obtention d'une adresse IP valide confirme le succès de la liaison sans fil, étape logicielle indispensable avant de pouvoir commencer à intercepter les paquets de données d'éclairage).

4. Outils

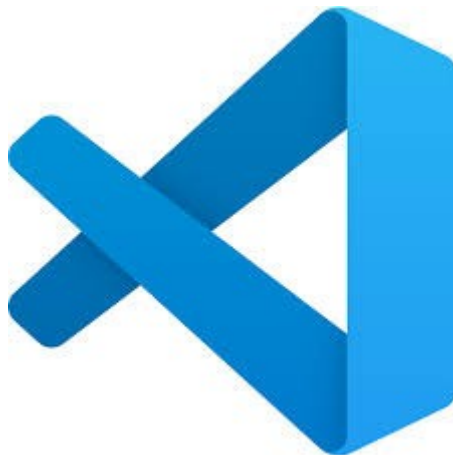
4.1. Visual Studio Code (VS Code)

Visual Studio Code est un éditeur de code source extensible, léger et multiplateforme développé par Microsoft. Il est particulièrement apprécié par les développeurs pour sa modularité et sa capacité à s'adapter à quasiment tous les langages de programmation grâce à un système d'extensions très complet.

Dans le cadre de ce projet de passerelle DMX, VS Code a été choisi comme Environnement de Développement Intégré (IDE) principal pour l'écriture de l'ensemble du code source en C++.

Visual Studio Code est utilisé pour :

- Centraliser le développement : Offrir une interface unique pour écrire, organiser et naviguer entre les différents fichiers d'en-tête et sources (`main.cpp`, `RecepteurDMX.h`, `ProjecteurLED.h`, etc.).
- Améliorer la lisibilité du code : Grâce à la coloration syntaxique, l'autocomplétion intelligente (IntelliSense) et la détection d'erreurs en temps réel avant même la compilation.
- Intégrer les outils de versionning : Gérer l'historique du code et les sauvegardes de manière fluide directement depuis l'éditeur.



HENRIETTE Naël (Étudiant 4)

4.2. PlatformIO

PlatformIO est un écosystème de développement professionnel "cross-platform" conçu spécifiquement pour l'Internet des Objets (IoT) et les systèmes embarqués. Intégré sous forme d'extension à Visual Studio Code, il remplace avantageusement des outils basiques comme l'IDE Arduino en offrant un contrôle absolu sur la chaîne de compilation (Toolchain).

Pour ce projet impliquant des protocoles stricts (Art-Net, DMX512) et des contraintes matérielles fortes sur l'ESP32, PlatformIO a été le moteur technique indispensable.

- **Reproductibilité via `platformio.ini`** : L'approche déclarative de PlatformIO centralise toute la configuration du projet dans un seul fichier. J'ai pu y définir la cible matérielle exacte (la carte ESP32), la fréquence du processeur, ainsi que la vitesse du moniteur série, garantissant que le projet se compile de manière identique sur n'importe quel ordinateur.
- **Gestion automatisée des dépendances** : PlatformIO gère le téléchargement et l'intégration des bibliothèques externes de manière isolée pour chaque projet. Cela m'a permis d'intégrer proprement des librairies complexes (comme celles nécessaires pour le ruban LED ou la communication I2C) en fixant leurs versions, évitant ainsi les conflits logiciels.
- **Compilation et débogage avancés** : L'outil optimise la compilation du code C++ et gère le téléversement (flashage) dans la mémoire de l'ESP32. Son moniteur série intégré a été crucial pour déboguer le flux réseau UDP et valider la bonne réception des trames Art-Net en temps réel.



4.3. Bibliothèques utilisées

Pour répondre aux contraintes matérielles et temporelles strictes du projet sans avoir à réinventer des pilotes de très bas niveau, l'architecture logicielle s'appuie sur plusieurs bibliothèques spécialisées. Leur intégration a été gérée de manière isolée via PlatformIO pour garantir la stabilité du code.

4.3.1. FastLED (Gestion du ruban lumineux)

FastLED est une bibliothèque open source de référence dans le domaine de l'embarqué pour le contrôle performant des rubans de LED adressables (comme le WS2812B). Contrairement aux bibliothèques basiques, elle offre un accès très bas niveau au matériel tout en proposant des fonctions mathématiques avancées. Pour ce projet nécessitant des animations fluides, FastLED a été le composant logiciel central de la classe `ProjecteurLED`.

- **Génération de signaux temporels stricts** : Le protocole de communication du ruban WS2812B est asynchrone et repose sur des micro-impulsions électriques extrêmement précises. FastLED gère cette contrainte matérielle en arrière-plan sur la broche `DATLEDS` (GPIO 19), sans saturer le processeur de l'ESP32.
- **Gestion de l'espace colorimétrique** : La bibliothèque permet de manipuler les couleurs via l'espace HSV (Teinte, Saturation, Valeur) plutôt qu'uniquement en RGB. Cela m'a permis de coder très facilement des effets dynamiques, comme la boucle de teinte globale (`gHue`) utilisée dans l'animation *Rainbow*.
- **Manipulation de la mémoire (Buffer)** : FastLED permet de travailler sur un tableau mémoire représentant virtuellement les LED (`CRGB leds[NB_PIXELS]`). J'ai pu ainsi diviser le ruban en plusieurs zones indépendantes et y appliquer des effets différents avant d'envoyer l'ordre d'affichage global.

4.3.2. Bibliothèque `esp_dmx` (Contrôleur RS-485)

La gestion du DMX512 requiert une précision que les fonctions de communication série standards de l'ESP32 ne peuvent pas garantir à elles seules. La bibliothèque `esp_dmx` a été choisie car elle interagit directement avec les registres matériels du microcontrôleur pour écouter un signal conforme aux normes de l'industrie du spectacle.

- **Exploitation matérielle de l'UART** : La bibliothèque prend le contrôle total du port série matériel et gère les vitesses de transmission (250 000 bauds), les bits de stop, ainsi que le signal critique de début de trame DMX (*Break* et *Mark After Break*). Elle s'interface parfaitement avec notre puce MAX485 via les broches `RX_RS232` et `TX_RS232`.

- **Réception et extraction asynchrones** : Intégrée dans ma classe `RecepteurDMX`, la bibliothèque place un processus d'écoute en tâche de fond. Lorsqu'un univers DMX est détecté, elle le place de manière sécurisée dans un tableau de 513 octets (`dmxData`), me permettant d'extraire la valeur précise de n'importe quel canal de manière instantanée.

4.3.3. Bibliothèque SSD1306 (Interface Homme-Machine)

Pour la restitution visuelle des informations de la passerelle en temps réel, le choix s'est porté sur la bibliothèque SSD1306. Elle est spécialement conçue pour piloter les écrans OLED monochromes via le bus I2C, en alliant légèreté en mémoire et rapidité d'exécution.

- **Héritage et personnalisation (POO)** : Au lieu de simplement appeler les fonctions de la bibliothèque, je l'ai utilisée comme base structurelle. Ma propre classe `EcranOLED` hérite directement de la classe mère `SSD1306`. Cela m'a permis de bénéficier de toutes les fonctions de dessin natives tout en créant mes propres méthodes métier (comme `afficherStatutWifi` ou `actualiser`).
- **Communication I2C non bloquante** : La bibliothèque gère la communication sur les broches SDA (21) et SCL (22) à haute vitesse (400 kHz). Associée à mon propre système de minuteur interne, elle a permis de mettre à jour les statistiques de l'écran sans provoquer de ralentissement global du programme.

4.3.4. WiFi et WiFiUDP (Connectivité réseau et Art-Net)

Pour transformer l'ESP32 en véritable nœud réseau, le projet s'appuie sur les bibliothèques réseau natives du framework ESP32 (`WiFi.h` et `WiFiUDP.h`). Elles masquent la complexité de la pile TCP/IP et permettent de dialoguer directement avec le routeur de la régie lumière.

- **Gestion de la couche réseau (Mode STA)** : La bibliothèque `WiFi` a été utilisée pour authentifier la carte sur le réseau local (SSID et mot de passe définis dans `config.h`). Elle gère de manière autonome la négociation DHCP pour obtenir une adresse IP valide, étape indispensable avant toute réception de données.
- **Traitement des datagrammes (UDP)** : Contrairement au TCP qui est lourd et exige des accusés de réception, la bibliothèque `WiFiUDP` permet d'ouvrir un port d'écoute ultra-rapide (le port standard Art-Net 6454). Elle est utilisée dans la boucle principale pour intercepter les paquets bruts entrants à la volée, vérifier leur taille, et extraire l'entête Art-Net et les valeurs lumineuses sans aucune latence perceptible.

4.3. Github

GitHub est utilisé comme plateforme de gestion de version et de collaboration. Cet outil s'est révélé particulièrement utile pour organiser, suivre et sécuriser le développement du code tout au long du projet. L'utilisation de GitHub présente plusieurs avantages concrets :

- Historique des modifications : chaque changement apporté au code source est enregistré, ce qui permet de revenir à une version antérieure en cas d'erreur ou de dysfonctionnement.
- Travail collaboratif : GitHub facilite le travail en équipe grâce au partage de dépôts, à la gestion des branches, et à l'intégration de contributions multiples sans conflit. Sauvegarde sécurisée : le code est stocké en ligne, évitant toute perte liée à une défaillance locale.
- Professionnalisation : GitHub est un outil couramment utilisé dans le milieu professionnel. Son utilisation dans le cadre de ce projet permet d'acquérir de bonnes pratiques de développement logiciel.

Grâce à GitHub, la gestion du code a été plus fluide, plus structurée et mieux adaptée à la collaboration entre les membres du projet. Cela a contribué à la réussite technique et organisationnelle du pilotage de l'éclairage scénique en Art-Net et DMX512.

Le lien pour accéder à notre projet final et nos dossiers (commun et personnels) se trouve ici :

https://github.com/aallard12/project_DMX_Art-Net/tree/nhenriette



5. Diagrammes de classe

Diagramme de Classe - Cas d'utilisation 1 (Passerelle Réseau/DMX)

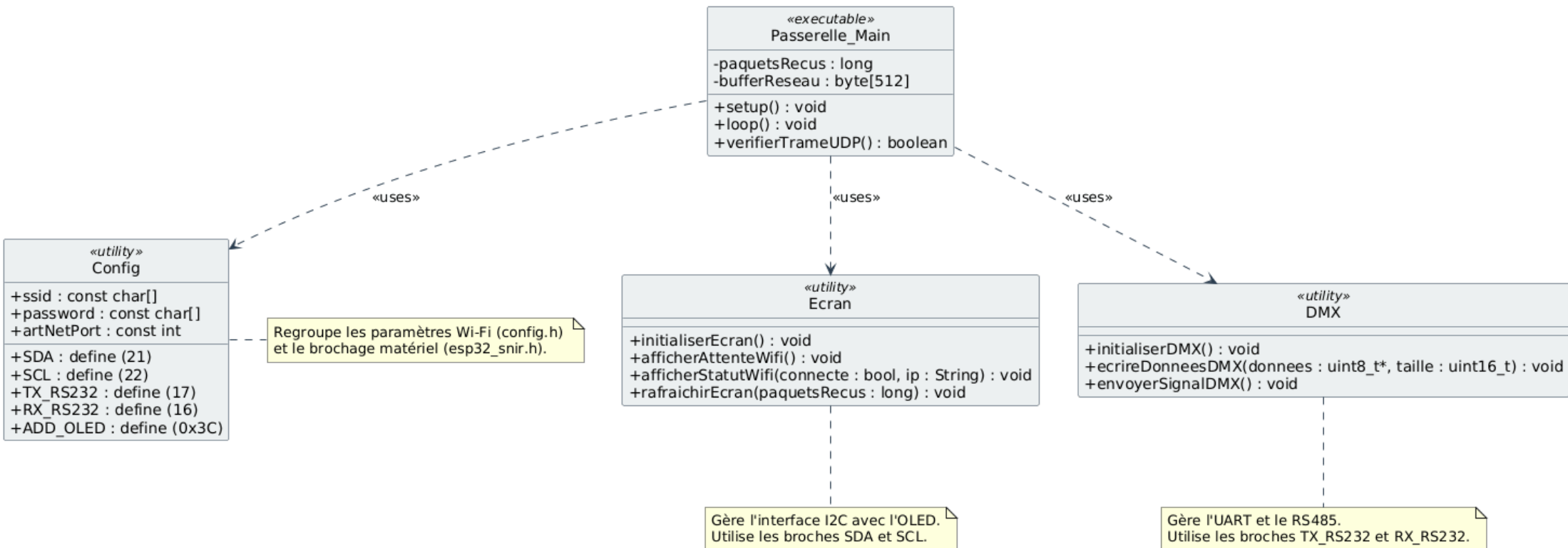
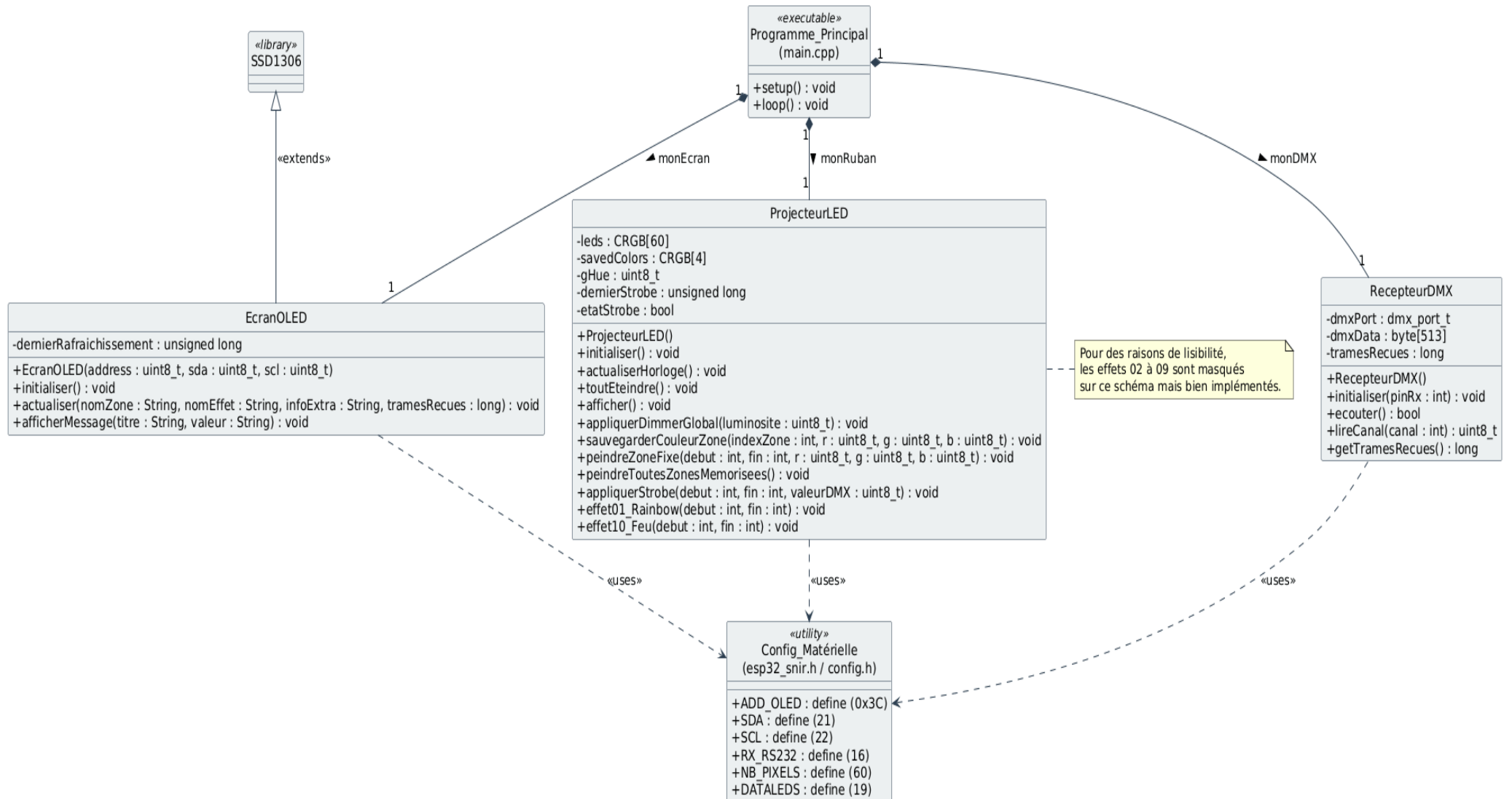


Diagramme de Classe - Cas d'utilisation 2 (Contrôleur Local LED & DMX)



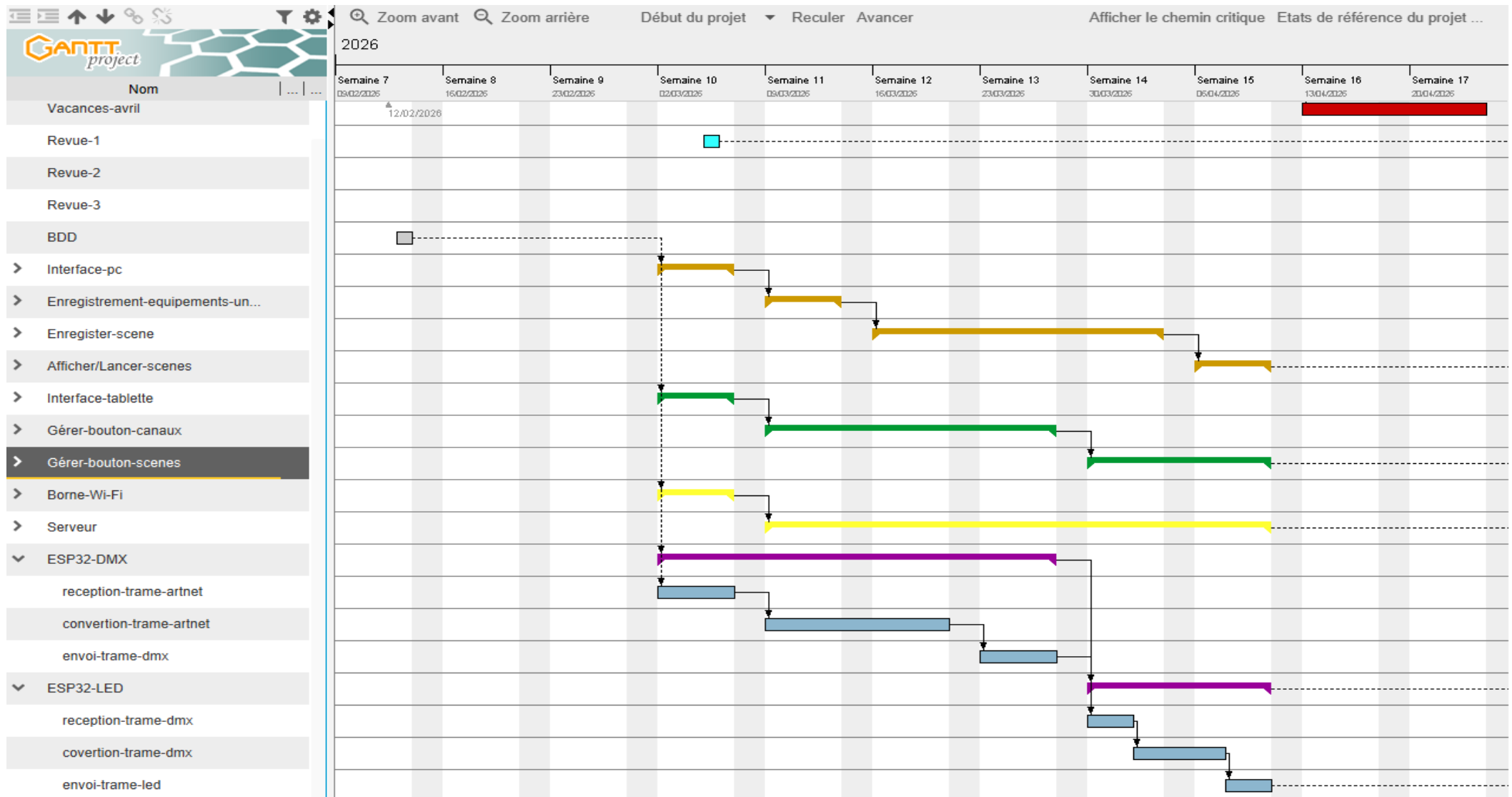
6. Fiches de test

Référence du module testé :			F4.1
Date du test :	30 avril 2026		
Type du test :	Test fonctionnel		
Objectif du test :	Valider le bon fonctionnement du cas d'utilisation « Piloter le ruban LED à partir du protocole DMX512 »		
<u>Conditions du test</u>			
État initial • La carte ESP32 est sous tension. • L'adresse DMX de départ a été saisie et validée au clavier (ex: Adresse = 1). • Le ruban LED est éteint (tous les canaux DMX sont à 0). • L'écran OLED affiche l'interface principale en attente de signal.		Environnement du test • Matériel : Maquette ESP32 complète (MAX485, Écran OLED I2C, Clavier 4x3, Ruban LED WS2812B). • Logiciel : Ordinateur avec IDE Arduino (Moniteur Série ouvert à 115200 bauds). • Réseau/Contrôle : Logiciel de console DMX/Art-Net configuré pour émettre.	
<u>Procédures du test</u>			
N°	Opérations	Résultats attendus	

1	- Ouvrir le projet dans visual studio code qui se trouve ici : /home/USERS/ELEVES/CIEL2/024/nhenriette/Documents/ciel2/projet_fin_dannee/ruban_led_v2 - brancher l'esp32 à un port USB sur le pc - ouvrir le monitor série sur le projet avec l'icône de prise électrique en haut à droite de la page - appuyer sur le bouton « en » sur le coté de l'esp32 pour la redémarrer et avoir les infos de test qui s'affiche à son démarrage	L'écran OLED s'allume. Le ruban LED flashe brièvement en Rouge, Vert, puis Bleu (test matériel). Le moniteur série affiche un bilan positif du diagnostic mémoire (4/4 tests réussis).
2	Validation de l'IHM (Clavier Matriciel) : L'écran OLED affiche "<i>Saisir Adresse DMX</i>". Taper le chiffre 1 puis valider avec la touche # du clavier.	La Zone 1 du ruban LED (les 15 premières LEDs) s'allume instantanément en blanc pur. L'écran OLED affiche Z : Z1 (1-15).
3	Test d'éclairage basique (Couleur) : Sur le logiciel DMX, mettre les Canaux 1, 2 et 3 (Rouge, Vert, Bleu) à la valeur 255. Laisser le Canal 4 (Zone) et Canal 5 (Mode) à 0.	La Zone 1 du ruban LED (les 15 premières LEDs) s'allume instantanément en blanc pur. L'écran OLED affiche Z : Z1 (1-15) et le moniteur série confirme la réception.
4	Test de routage spatial (Zones) : Monter le Canal 4 (Zone) à la valeur 75. Garder les couleurs à 255.	La Zone 1 s'éteint et la Zone 2 (LEDs 16 à 30) s'allume en blanc. L'écran OLED met à jour l'affichage avec Z : Z2 (16-30).

5	Test de commutation de mode (Stroboscope) : Monter le Canal 5 (Mode) à la valeur 200.	La zone sélectionnée sur le ruban LED se met à clignoter rapidement (Stroboscope). L'écran OLED met à jour son statut : FX: STROBE.
6	Test des algorithmes d'effets (CH6) : Pousser le Canal 5 (Mode) à 255. Mettre le Canal 6 (Effets) à la valeur 20.	Le ruban LED lance l'animation dynamique "Arc-en-ciel" (Rainbow). L'écran OLED affiche FX: EFFETS (CH6) 20.
7	Test du Mode Espion (Console de débogage) : Modifier légèrement une couleur sur le logiciel DMX et observer le Moniteur Série de l'ordinateur.	Le moniteur série affiche instantanément --- NOUVELLE COMMANDE DETECTEE --- et détaille le nombre de trames reçues ainsi que la conversion des valeurs.
8	Test de sécurité (Failsafe / Perte de signal) : Débrancher le câble réseau/DMX (ou couper l'émission sur le logiciel PC) et attendre 3 secondes.	Le ruban LED s'éteint complètement (Blackout de sécurité). L'écran OLED affiche le message d'erreur : ALERTE : SIGNAL DMX PERDU !.

7. Diagramme de Gantt



8. Compétences acquises

Sur le plan technique :

- J'ai vraiment progressé en C++ orienté objet. Au début, structurer mon code en séparant les fichiers (.h et .cpp) et comprendre des concepts comme l'encapsulation (attributs privés/publics) ou l'héritage me paraissait abstrait.
- Faire communiquer le monde de l'informatique réseau et celui de l'électronique matérielle m'a demandé du temps à assimiler. Mais voir la passerelle traduire instantanément une trame Wi-Fi (UDP/Art-Net) en un signal électrique différentiel (RS-485) pour allumer un projecteur a été une vraie satisfaction
- J'ai appris à abandonner les simples `delay()` qui bloquent le processeur pour concevoir un code totalement asynchrone. Gérer l'écoute réseau, le ruban LED et le rafraîchissement de l'écran OLED en parallèle grâce à des chronomètres internes a été un vrai cap technique franchi
- L'utilisation d'outils de développement de niveau professionnel comme Visual Studio Code couplé à PlatformIO m'a montré la différence avec un IDE classique, notamment pour la gestion rigoureuse des dépendances matérielles et de la compilation
- Les protocoles de l'industrie du spectacle (DMX512 et Art-Net) étaient complètement nouveaux pour moi en début de projet. Les décortiquer pour extraire uniquement les octets utiles m'a appris à analyser la structure brute d'une trame réseau

Sur le plan professionnel :

- J'ai appris à ne pas foncer tête baissée dans le code, à prendre le temps de concevoir les cas d'utilisation et les diagrammes en amont m'a évité beaucoup de problèmes
- Maintenir une qualité de code lisible tout au long du projet n'est pas aussi naturel qu'on le croit, ça demande de la discipline
- Rédiger une documentation et un dossier technique compréhensibles par quelqu'un qui ne connaît pas le projet m'a forcé à prendre du recul sur mon propre travail pour aller à l'essentiel
- Rédiger une documentation compréhensible par quelqu'un qui ne connaît pas le projet m'a forcé à prendre du recul sur mon propre travail

En conclusion, ce projet m'a beaucoup apporté, autant techniquement qu'humainement. Je repars avec des compétences concrètes en développement embarqué que je n'avais pas en début d'année, et surtout avec une méthode de travail solide que je compte bien conserver et appliquer à l'avenir.

9. Perspectives d'améliorations

- **Configuration Wi-Fi dynamique via portail captif** : aujourd'hui, les identifiants réseau (SSID et mot de passe) sont écrits en dur dans le fichier `config.h`. Implémenter un mode Point d'Accès temporaire (type Wi-Fi Manager) permettrait à l'utilisateur de configurer la passerelle depuis son smartphone lors d'un changement de salle de spectacle, sans avoir à recompiler et téléverser le code.
- **Mise à jour du firmware à distance (OTA)** : le code est actuellement flashé via la connexion physique USB. Ajouter la fonctionnalité Over-The-Air permettrait de mettre à jour le programme de l'ESP32 directement par le réseau Wi-Fi, ce qui serait un gain de temps énorme et indispensable une fois le boîtier installé en hauteur dans des ponts d'éclairage.
- **Personnalisation du mode Sécurité (Failsafe)** : le système coupe actuellement toute la lumière (Blackout) en cas de perte du signal Art-Net pendant 3 secondes. Proposer d'autres comportements, comme le maintien en mémoire du dernier état lumineux ("Hold") ou le lancement automatique d'une animation d'attente douce sur le ruban local, rendrait le système plus flexible en cas de panne réseau.
- **Sauvegarde des paramètres en mémoire flash** : utiliser le système de fichiers de l'ESP32 (NVS ou SPIFFS) permettrait de retenir le dernier effet local choisi ou le dernier numéro d'Univers Art-Net assigné. Cela éviterait que le système ne reprenne ses valeurs par défaut à chaque coupure électrique.
- **Surveillance thermique et électrique** : l'écran OLED affiche déjà le statut réseau et les trames, mais ne surveille pas le matériel. Ajouter la lecture de la température interne du processeur ou surveiller la tension d'alimentation permettrait d'afficher des alertes préventives sur l'écran avant une défaillance matérielle en plein show.

Fonctionnalité phare :

- **Interface Web de supervision embarquée** : la passerelle fait son travail de conversion, mais n'offre pas de retour visuel global en dehors du petit écran OLED. Héberger un serveur Web asynchrone directement sur l'ESP32 permettrait au technicien d'ouvrir une page web depuis son ordinateur pour visualiser l'état des 512 canaux DMX en temps réel, changer l'univers d'écoute, ou forcer manuellement le ruban LED pour faire des tests sans avoir besoin d'allumer la console de régie.

En conclusion, la passerelle est aujourd'hui fonctionnelle et couvre l'ensemble des besoins du cahier des charges. Ces améliorations ne remettent pas en cause ce qui a été réalisé, mais elles viendraient compléter et solidifier le système matériel pour une utilisation professionnelle et autonome sur le long terme.